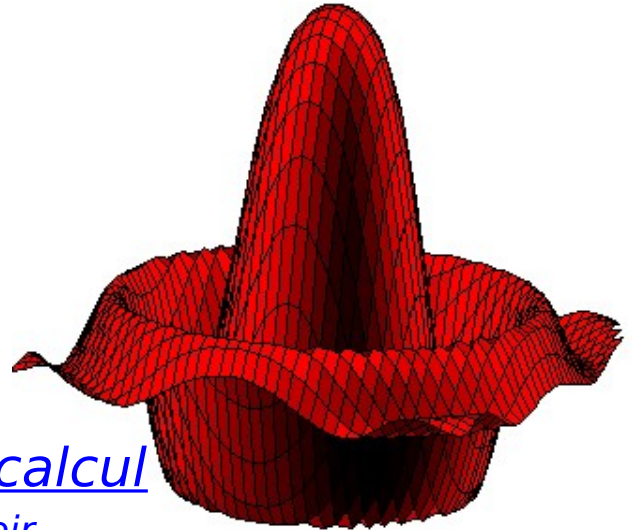


DOCUMENTATION CALCREG V1.13.6

®Copyright 2013 BUSSY-SOCRATE Régan



I. [Licence](#)

II. [Historique](#)

III. [Rouage interne de calcul](#)

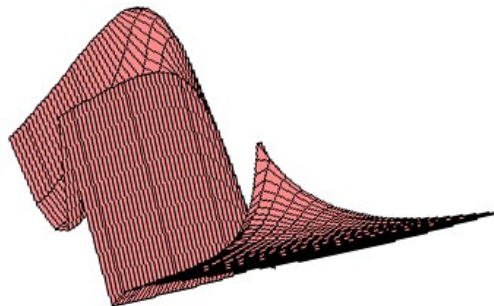
- [Ce qu'il faut savoir](#)

IV. [Jeu d'Instructions et Règles de Calcul](#)

- [Préprocesseur](#)
- [Testes logiques et branchement et sous-routines](#)
- [Fonctions et Tracés de Graphs](#)
- [Calcul Matriciel](#)
- [divers](#)

V. *Bugs connus.*

VI. *Information et contact*



I) Licence

Ce logiciel est sous licence GPL-v3. Si vous souhaitez contacter l'auteur voici l'email reg.bussy.s@gmail.com .

En utilisant ce logiciel, vous prenez l'entière responsabilité de ce qui peut arriver à votre ordinateur. En aucun cas, l'auteur de ce logiciel ne sera tenu pour responsable de ce qui peut arriver autant sur le support matériel de votre ordinateur que pour la partie soft (mémoire, perte de données, défaillance logicielle).

Par exemple : Si vous utilisez ce logiciel pour lancer un satellite en orbite et que la trajectoire décroche, vous ne pourrez pas tenir l'auteur de ce logiciel pour responsable.

II) Historique

Ce logiciel est né du besoin d'avoir un outil mathématique pour manipuler les graphes très aisément sur un PDA. Le cahier des charges était : un tracé simple comme on pourrait l'écrire sur le cahier, un déplacement des courbes avec le stylet. L'utilisation de nombres complexes, la possibilité d'utiliser le port série du Palm pour effectuer des mesures de tension et traçage de l'oscillographe. Cela donna en 2012 CalcRegV1.9 pour PalmOs.

Comme le PalmOs était assez limité en vitesse et en mémoire, l'idée est venue de le porter sur PC pour une utilisation plus massive. Pour en faire un petit laboratoire de mesures et de traitement de données. En électronique, l'utilisation des nombres complexes est essentielle, avec l'usage du calcul matriciel et des tableaux de nombres dans le même temps. Tracé de Courbes , d'objets en 2D et 3D. Traitement du son, comme base de traitement du signal.

Le portage du PalmOs vers le PC n'a pas été si facile, l'utilisation de Win32 API pourtant simple amenait à trouver des solutions frustrante et parfois instable. Mais le maniement des courbes et l'interactivité avec le programme a pu être maintenu par rapport au portage sur XP, Vista et win7 donnant un outils

utilisable et assez rapide par rapport à ce que l'on trouve dans ce domaine de logiciel.

III) Rouage interne de calcul

a) **La librairie mathématique.**

Originellement, le programme a été bâti pour le PalmOs, qui n'avait pas de librairie mathématique légère à utiliser. Mathlib.prc est assez lourde et donc lente. La solution a alors été de rebâtir toutes les fonctions mathématiques à partir d'algorithmes les plus rapides possibles, avec sur le PalmOs la possibilité de régler la précision de calcul attendue. Comme cela fonctionnait plutôt bien, ces algorithmes ont été conservés.

Ultérieurement, la possibilité de passer exclusivement à l'utilisation des librairies systèmes sera offerte sur CalcReg. Les librairies systèmes peuvent être légèrement plus lentes mais l'algorithme est confirmé et plus sûr. Cette option de calcul n'existe pas encore dans CalcReg.

b) **Les nombres complexes.**

A un moment donnée, l'utilisation d'un tel logiciel de calcul nécessite des connaissances en mathématique qui peuvent être plus ou moins poussées. Dans ce qui suit, nous entrons dans le détail donc.

CalcReg présente une option pour activer ou bien désactiver le mode de calcul avec les nombres complexes. Le calcul **matriciel avec les nombres complexes** est présent dans CalcReg à partir de la version 1.13.6. Bien sûr si vous n'utilisez pas les nombres complexes dans votre programme, vous souhaitez qu'il tourne le plus vite possible. D'où l'option d'activer ou non les nombres complexes dans le programme. Cette activation se fait par le Préprocesseur `//#COMPLEXE`, ou bien à l'aide du menu dans `MODE/Toggle Allow Complexe`.

' i ' est donc le nombre complexe tel que $i^2 = -1$. La Majuscule de i n'est pas reconnue comme le nombre complexe i. Si vous

n'activez pas les nombres complexes, i sera pris pour une variable. Si cette variable n'est pas initialisée par 'i=...' alors cela génèrera une erreur. Donc chaque fois que vous utilisez les complexes, commencez plutôt par mettre en entête de votre programme le préprocesseur `///COMPLEXE`.

c) ***Les priorités de calcul et la boucle centrale interne.***

La boucle centrale est Le Main du programme de CalcReg.

Lorsque l'on appuie sur le bouton EXE, le programme présent dans la fenêtre est d'abord converti en Codes. Ce ne sont pas des codes du langage machine. Ce sont des codes pour CalcReg en interne qui portent l'instruction et l'opérande. Cette liste de codes sera alors exécutée dans un second temps. Ce n'est donc pas un langage compilé. Il est optimisé pour le calcul.

Le convertisseur en Code vérifie donc la syntaxe, convertit les instructions en codes, et effectue la conversion des variables en Accumulateurs A0-...Amax (Amax est fixé lors de la compilation de CalcReg). C'est-à-dire que $x=1$ deviendra $A0=1$ ou bien $A12=1$... le numéro de l'accumulateur correspond à sa place dans la liste réorganisée par CalcReg. Cette remarque est essentielle si l'on veut débbugger son programme et que l'on tombe sur une liste de mnémoniques $A0=1$, $A1=A0+2*A3$,... par exemple. Donc évitez l'utilisation de variable avec A(..) telles que A0,A1,A2,... Il pourrait en résulter une erreur très difficile à repérer. Vous pouvez librement utiliser a0, a1, x0, X0,x1,X1,...

Une fois la CodeList obtenue, le programme lance la boucle qui va exécuter chaque ligne et les branchements, les syntaxes de calcul, avec les opérations à effectuer.

L'analyse de la syntaxe de calcul est le point de départ d'un tel logiciel. Il faut qu'elle soit rapide et respecte les priorités mathématiques. D'abord les parenthèses, puis les fonctions, les

fonctions puissances et ensuite produit '*', la division '/', puis la somme '+' et la soustraction '-'.

Donc le sens de l'analyse de la chaîne de calcul $x=1+x^2*2/((1+1/2*\cos(x)))$ va se faire comme exprimé ci-dessous :

0) **Priorité parenthèse** : Prendre le calcul qui est au plus profond des parenthèses.

1) **Remplacer les variables par leur valeur** : Placer la valeur de la variable x à la place de x après le signe =. Imaginons que $x=4$, on s'en sert dans la suite.

2) **Calcul des fonctions mathématiques** : Effectuer l'opération de la fonction $\cos(4)$ et placer résultat à la place de $\cos(x)$.

3) **Chercher dans l'ordre '*' '/' puis '+' et '-'** : $2*\text{ans}$, puis $1/\text{ans}$ et $1+\text{ans}$ où ans est le précédent calcul.

4) La parenthèse étant traitée, la priorité la plus élevée est la puissance 2 . Et ensuite les règles de priorité $*/+-$.

d) ***Ce qu'il faut savoir.***

Pour utiliser une variable quelconque, il faut qu'elle soit reconnaissable par le convertisseur en Codes. Une variable est donc considérée en tant que telle à partir du moment où elle est définie quelque part dans le programme par `var=...`

<u>Exemple :</u>	<u>Contre-Exemple :</u>
<code>gfxdim -1,1,-2,10,1</code>	<code>gfxdim -1,1,-2,10,1</code>
<code>delta=1</code>	<code>delta=1</code>
<code>x=-1</code>	<code>Trf(x)=delta*x+1</code>
<code>Trf(x)=delta*x+1</code>	<code>end</code>
<code>end</code>	

Dans ce programme, tout est bien défini. X et delta seront reconnu par le convertisseur.	Dans ce programme, il manque $x=-1$, donc une erreur sera générée lors de l'exécution de $\text{Trf}(x)$,... car le code pour x n'indiquera pas qu'il s'agit d'une variable.

IV) Jeu d'instructions et règles internes

Le jeu d'instruction est fait tel que, sauf cas exceptionnel rare, elles soient compatibles avec les versions précédentes de CalcReg.

PreProcessor instruction:

Les instructions en préprocesseur doivent être placées en début du programme. Elle disent à CalcReg que vous utilisez les complexe dans votre programme. Cela signifie que l'interprétation du programme dans sa totalité et son exécution se fera avec les réglages prédéfinis à partir des préprocesseurs.

//#COMPLEXE Active l'utilisation des nombres complexes. Vous pourrez utiliser des nombres complexes tels que $x=1+2*i$, vous pourrez utiliser des matrices complexes comme le montre l'exemple ci-dessous :

```
//#COMPLEXE
```

```
defM1,2,3           //definit une matrice appelée M1 de dimension 2x3
```

```
dataM1=i,2,(3+i),5,(2*i),1 //remplit M1 avec les Mnp dans le sens p croissant
```

```
print M1           //afficher les valeurs de la matrice
```

```
end                // arrêter le programme.
```

//#NOCOMPLEXE Désactive l'utilisation des complexes, on n'utilisera que les réels. 'i' sera considéré comme une variable à initialiser.

Remarque : Si aucun préprocesseur sur les complexes n'est présent, le lancement du nouveau programme s'effectuera avec le réglage du précédent lancement de programme.

Pour l'usage des nombres complexes avec le calcul matriciel, un bon exemple de programme est la simulation du filtre Butterworth à 5 niveau voir dans le répertoire BUTTERWORTH FILTER / Butterworth Bode diagram 2.txt

Jeu d'instructions:

Teste logique et Branchements et labels:

Label:	Les labels pour les branchements se notent NOMDULABEL :
goto label	<i>Effectuer un saut. Exemple : goto loop.</i>
bsr label	Branchement en sous routine.
rts	<i>Retour de sous routine.</i>

a < b	<i>Teste de condition : Si a est inférieur à b.</i>
a > b	<i>Teste de condition : Si a est supérieur à b.</i>
a == b	<i>Teste de condition : Si a est égal à b.</i>
=> <i>Exemple :</i> <i>h=0</i> <i>loop :</i> <i>print h</i> <i>h=h+1</i> <i>h<10 => goto</i>	<i>Conséquence : Alors</i> <i>Le programme ci-contre permet d'effectuer une boucle de comptage jusqu'à 9 et d'imprimer chaque chiffre de 0 à 9.</i>

loop end	
-------------	--

Tracé de graphes, fonctions et instructions graphiques

grid Step	Step est le pas pour les graduations des axes x et y du repère.
gfxdim xmin,xmax,ymin,ymax,IncX	<i>Les dimensions du repère et l'incrément IncX, pour l'affichage des fonctions f(x) sur le graphe.</i>
box3d wx,wy,wz,Inc3d	<i>wx dimension de la boîte en x de -wx à wx. idem pour wy et wz. Inc3d est le pas d'avancement pour la finesse du dessin. Exemple d'utilisation voir Trf(x,y).</i>
line x1,y1,x2,y2,color	<i>Trace une ligne dans le repère de dimension définie par gfxdim. La ligne relie les point P1(x1,y1) et P2(x2,y2)</i>
colorgfx color	Fixe la couleur du trait : Color=0 noir Color=1 rouge Color=2 vert Color=3 bleu
gfxdim -1,1,-5,5,0.1 x=-1 Trf(x)= 2*x+1	<i>Trf(x)= est l'instruction qui permet de tracer une fonction sur le graphe de dimension définie par gfxdim. Si gfxdim n'est pas redéfini, Trf(x) se tracera sur la dimension définie précédemment dans le programme. N'oubliez pas de mettre dans la variable (ici x) la valeur de démarrage du graphe. Dans l'exemple ci-contre, si x=2, le graphe ne se tracera pas car $x=2 > 1$ du fenêtrage de gfxdim.</i>

	Si l'on fait plusieurs tracés successifs, la couleur de la courbe change automatiquement. Mais on peut sélectionner une couleur particulière pour le prochain tracé par l'instruction colorgfx.
<pre>gfxdim -7,5,-4,5,0.2 box3d 4,4,4,0.08 x=4 y=-4 Trf(x,y)=3*exp(-(x^2+y^2)/ 2.5)*cos(0.5*(x^2+y^2))</pre>	En 3D on utilise Trf(x,y) où x et y sont des variables quelconques préalablement initialisée par x=xmax et y=ymin. Le tracé de la fonction 3D dans ce cas utilise une routine qui trace par lignes pour un rendu 3D. voir box3d pour le réglage du pas de dessin (ici 0.08) Il est possible de tracer une fonction 3D en passant par la création d'un objet 3D et de le tracer avec l'instruction <code>dispobj</code>. Dans ce cas le rendu est plein.
<pre>F1(x) = 8*x+2 Exemple : //Tracer la dérivée de F1(x) F1(x) = 5*x^2+2 h=0.00001 Trf(x)= (F1(x+h)-F1(x-h))/h</pre>	$F(.)$ est la fonction définie par ce qui suit. Une fois définie, elle se comportera et s'utilisera comme une fonction mathématique utilisable telle que $\cos(x)$ par exemple. Par contre, l'utilisation avec les matrices est encore à implémenter.
<pre>print x //affichage valeur de x printx,"Text x=" // Text= valeur(x) print M1 //affichage matrice M1</pre>	Pour afficher la valeur de x dans la fenêtre cmd du bas. Si l'on ne met pas de texte à afficher avec, on peut mettre un espace entre <code>print</code> et la variable(ici x). Mais on met du texte entre guillemet, alors x doit être collé contre <code>print</code> sans espace. C'est un détail qui sera corrigé dans les versions future.

Calcul Matriciel:

Quelques notes sur le calcul matriciel et la syntaxe :

Les matrices se nomment $M0, M1, M2, \dots$ jusqu'à $M_{\max}$ défini par CalcReg en interne. Il y a les matrices fixes, et les matrices utilisées en interne pour les intermédiaires de calcul.

Au sens rigoureusement mathématique, le calcul matriciel se présente selon la syntaxe : $M4 = a * M0 + b M1 + c M2 * M3$

a, b et c sont des scalaires, $M0, \dots, M4$ sont des matrices de dimensions compatibles avec le calcul matriciel mathématique. Donc $M0_n = M1_n = M4_n = M2_n$, $M0_p = M1_p = M4_p = M3_p$ et $M2_p = M3_n$.

L'interprétation de la syntaxe qui suit n'est pas conforme aux mathématiques, mais elle est très utile pour effectuer des calculs avec une masse de données, pour le traitement du signal par exemple. La condition d'interprétation de cette syntaxe par CalcReg, c'est que les matrices ont pour dimension $n=1$. Donc incompatibles avec le calcul matriciel rigoureux :

$$M4 = a + b * M0 + c * \sin(d * M0 + e * M0 * M0)$$

Dans ce cas, chaque matrice doit être de dimension $1 \times p$. $M4$ sera remplie par le calcul pour chaque élément comme un tableau de nombre. Et les scalaires sont considérés comme des matrices $1 \times p$ comme les autres. La détection de cette syntaxe est automatique à partir du moment où le calcul n'est pas possible au sens mathématique rigoureux.

D'une manière générale, si $F(x)$ est une fonction quelconque $\cos, \sin, \tan, \exp, \dots$ $F(Ma)$ est toujours considéré comme la fonction appliquée à chaque élément de la matrice Ma . Par exemple, $\cos(M0)$ est :

$\cos(M0(1,1)),$	$\cos(M0(1,2)),$
$\dots, \cos(M0(1,p))$	
$\cos(M0(2,1)),$	$\cos(M0(2,2)),$
$\dots, \cos(M0(2,p))$	

```
...
cos (M0(n,1)), cos(M0(n,2),
...,cos(M0(n,p)
```

CalcReg ne donnera pas le calcul du développement du cosinus en série de Taylor de M0, et cela pour toutes les fonctions cos, sin ,tan, exp,... appliquées aux matrices.

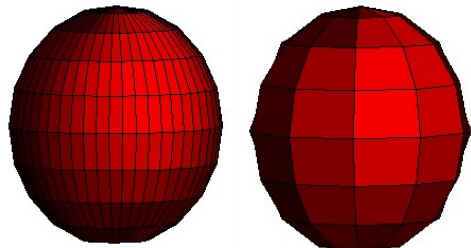
<code>print M1</code>	prints in cmd window the matrix only 10 by lines. a (...) code is displayed at the end of 10 data exceeding lines discarding the rest
<code>defM a,n,p</code> <i>Exemple :</i> <code>defM0,1,4 //définit M0 de dimension 1x4</code>	<i>Définition d'une matrice Ma de dimension nxp initialisée à 0.</i> <i>Si on réutilise le même numéro de matrice, alors la mémoire pour cette matrice est libérée et une nouvelle est allouée avec les nouvelles dimensions.</i>
<code>fillM n,a,b</code> <i>exemple :</i> <code>defM2,1,4</code> <code>fillM2,1.5,3.5</code> <code>print M2</code> <code>end</code>	<i>Remplit la matrice Mn avec des valeurs régulièrement espacée entre a et b inclus.</i> <i>Dans l'exemple : M2 est rempli avec les valeurs successives 1.5, 2.266.., 2.833.., 3.5 l'espacement est de (3.5-1.5)/(4-1)=2/3</i> <i>Pour obtenir 3.5 en dernière valeur, il faut l'espacement (b-a)/(N-1).</i>

<code>mtxn(Ma)</code>	<i>Fonction donnant la dimension n d'une matrice Ma de dimension nxp.</i>
<code>mtxp(Ma)</code> exemple : <code>defM1,1,3</code> <code>a=mtxp(M1)</code> <code>print a</code> <code>end</code>	<i>Fonction donnant la dimension p d'une matrice Ma de dimension nxp.</i> <i>Dans l'exemple ci-contre, le programme donne 3.</i>
<code>trp(Ma)</code> Exemple : <code>M1=trp(M0)</code>	<i>trp(Ma) retourne la matrice transposée de Ma.</i>

Matrices et audio

<code>loadsndMa</code> <code>loadsndMa,"myfile.wav"</code> exemple : <code>Nsp=44100</code> <code>loadM1,"monfichier.wav"</code> <code>playsndM1,Nsp //joue le son</code> à 44100 Sp/s	<i>Charge dans la matrice Ma un ou le fichier audio .wav où a est le numéro de la matrice concernée. La matrice est initialisée directement à partir des données du fichier.</i> <i>Dans l'instruction loadsndMa, une fenêtre s'ouvre et vous sélectionnez le fichier.</i> <i>C'est à vous de connaître le Sample rate car CalcReg n'a pas d'instruction à ce jour permettant de récupérer le nombre d'échantillon par seconde à partir du fichier .wav</i> <i>Dans la syntaxe loadM1,"monfichier.wav", ne faut pas mettre d'espace dans l'instruction sinon cela génère une erreur.</i>
--	---

<pre> playsndMa,SampleRate //mono playsndMa,sampleRate,b //stereo exemple : Nsp=44100p pi=3.1415927 defM1,1,Nsp defM2,1,Nsp defM3,1,Nsp fillM1,0,1 M2=8192*cos(2*pi*440*M1) M3=8192*cos(2*pi*880*M1) playsndM2,Nsp,3 end </pre>	<i>Jouer le son contenu dans la matrice Ma en mono ou Ma (left) et Mb (right) en stéréo. Ma et Mb doivent biensûr avoir même dimension.</i> <i>Dans l'exemple, le taux d'échantillonnage est de 44100 Sp/s. L'exemple génère un son stéréo : 440Hz sur la voie Gauche et 880 sur la voie de droite.</i> <i>Le signal est fabriqué selon la formule :</i> <i>$s(t)=8192*\cos(\omega t)$, avec $\omega=2\pi f$, avec f la fréquence du signal.</i> <i>M1 est la matrice contenant le temps t de 0 à 1(s). En 1s il y a Nsp échantillons, d'où la taille de M2 et M3.</i>
<pre> savesndMa savesndMa,"myfile.wav exemple : </pre>	Sauvegarde d'une matrice contenant les données d'un son en fichier audio.raw en mode mono au format 16bits. Il n'y a pas de possibilité de sauvegarder en stéréo à ce jour dans CalcReg.
<pre> recsndMa,Nsp exemple : Nsp=44100 defM0,1,(2*Nsp) //2*Nsp => 2s </pre>	<i>Enregistre un son dans une matrice Ma de taille nxp, au taux d'échantillonnage Nsp Sp/s. Datasize=n*p, si n=1, datasize =p.</i> <i>L'exemple enregistre un son à partir du micro, au taux de 44100Sp/s avec une qualité de 16bits.</i> <i>La durée d'enregistrement est fixée par</i>

<pre>recsndM0,Nsp //enregistrement playsndM0,Nsp end</pre>	<i>la taille de la matrice préalablement initialisée. Ici l'enregistrement dure 2s.</i>		
<pre>saveMa saveMa,"mtx.m"</pre>	<i>Sauve la matrice de numéro a. Si le nom entre guillemet n'est pas posé, une fenêtre Windows apparaît le nom du fichier et pour l'enregistrement.</i> <i>Remarque : Au niveau syntaxique, ne pas mettre d'espace, coller chaque caractère sinon une erreur est générée.</i>		
<pre>fillsphM a,PtLink,Radius,n</pre>  <table border="1" data-bbox="188 1352 683 1563"> <tr> <td> dimMa=3x50 0 n=50 </td> <td> dimMa=3x80 n=10 </td> </tr> </table> <pre>exemple : gfxdim -10,10,-10,10,1 N=500 //essayer 80 defM1,3,N defM2,4,N //Matrice Link</pre>	dimMa=3x50 0 n=50	dimMa=3x80 n=10	<i>Création d'un objet : une sphère de rayons Radius. La mémoire utilisée par l'objet est fixée à l'avance par la taille de Ma 3xp ou 4xp où p est le nombre des pts pour l'objet.</i> <i>a est le numéro de la matrice cible.</i> <i>PtLink est la matrice des liens. Chaque point est lié à quatre autres. La matrice MPtLink contient donc les indexes des points de lien dans Ma.</i> <i>La Matrice PtLink est dimension nxp=4x TotalPts.</i> <i>Ma(1,MPtLink(1,n)) est l'abscisse du point qui lie le point Px=Ma(1,n), Py=Ma(2,n), Pz=Ma(3,n)</i> <i>n contient le nombres de points sur un tour. Il faut que le nombre total</i>
dimMa=3x50 0 n=50	dimMa=3x80 n=10		

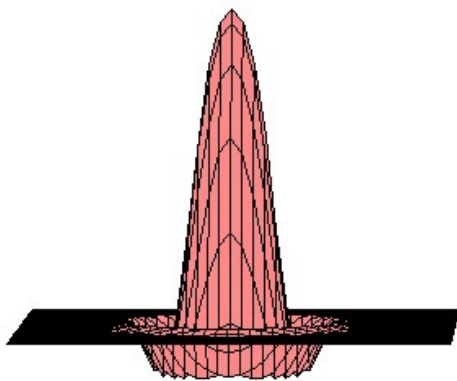
```
fillsphM1,2,5,50 //essayer 10
dispobjM1,2,3
end
```

de point soit un multiple de n .

Ma est de dimension $n \times p$ avec $n=3$ ou 4 . P est le nombre total de points. Il est parfois nécessaire d'avoir des objet avec une matrice $4 \times p$, alors qu'il n'y a que 3 coordonnées. C'est que le 4^{ème} est utilisé pour la translation de l'objet. On peut utiliser pour faire tourner l'objet et le translater une matrice 4×4 qui contient la translation. En utilisant un produit matriciel l'objet est tourné et translaté.

Vous n'avez pas vraiment besoin de savoir comment fonctionne la matrice MPtLink, mais elle est utile dans l'affichage de l'objet et autres fonction d'objet 3D.

dispobjM a,PtLink,Mode



exemple :

```
gfxdim -10,50,-30,30,1
```

```
action 0 //effacer l'espace
graphique
```

A est le numéro de la matrice objet contenant les coordonnées des points x,y,z et un nombre 1 pour la translation de l'objet.

Donc Ma de dimension $n \times p$, $n=3$ ou 4 et p égal au nombre de points constituant l'objet. PtLink est le numéro de la matrice contenant les indexes des 4 points autour faisant lien.

Mode est le mode d'affichage 3D de l'objet:

Mode:

<pre>x=1 y=1 N=40 m=N^2 defM0,N,N fM0(x,y)=5*N/8*exp(0.04*(-(x- N/2)^2-(y-N/ 2)^2))*cos(0.06*(-(x-N/2)^2- (y-N/2)^2)) defM1,4,m defM2,4,m fctobjM0,1,2//M1 objmtx,M2 ptlink dispobjM1,2,1 //mode 1 end</pre>	0= 3D fils de fer 1= Projection 3D dans un repère (x,y,z) avec x dirigé vers l'écran, y horizontal vers la droite, z verticale vers le haut. 2= Projection 3D sur 2D identique au dessin industriel. Il ne s'agit pas d'une perspective mais d'une vue 2D de l'objet. Vue de l'objet « écrasé » sur le plan y-z. 3= 3D avec lumière réfléchie voir l'instruction "action" pour fixer les coordonnées de la lampe.																		
<pre>dataMa,a11,a12,...,anp exemple: defM0,3,4 //matrice 3x4 dataM0,1,2,3,4 dataM0,5,6,7,8 printM0 end</pre>	Remplir la matrice avec des valeurs. a est le numéro de la matrice. Il est possible de faire le remplissage sur plusieurs lignes successives. Mais si l'on saut une ligne, on repart du début. <i>Le programme ci-contre donne :</i> <i>M0=</i> <table><tr><td>1.0e+000 </td><td>2.0e+000 </td><td>3.0e+000 </td></tr><tr><td>4.0e+000 </td><td></td><td></td></tr><tr><td>5.0e+000 </td><td>6.0e+000 </td><td>7.0e+000 </td></tr><tr><td>8.0e+000 </td><td></td><td></td></tr><tr><td>0.0e+000 </td><td>0.0e+000 </td><td>0.0e+000 </td></tr><tr><td>0.0e+000 </td><td></td><td></td></tr></table>	1.0e+000	2.0e+000	3.0e+000	4.0e+000			5.0e+000	6.0e+000	7.0e+000	8.0e+000			0.0e+000	0.0e+000	0.0e+000	0.0e+000		
1.0e+000	2.0e+000	3.0e+000																	
4.0e+000																			
5.0e+000	6.0e+000	7.0e+000																	
8.0e+000																			
0.0e+000	0.0e+000	0.0e+000																	
0.0e+000																			

<pre>//fMn°(x,y)= exp(- (x2+y^2)) fills the matrix with the exponential, it starts from x=1 (y=1) up to Mn (Mp respectively)</pre>	
<pre>//ftobjM n°M1,n°M2,n°PtLinkM transfert matrix style Meshgrid to an object with its PtLink Matrix to be displayable with dispobjM</pre>	
<pre>//getindM n°M1,n°VectP,n°IndexPts, Mask get the better index of point close to vectP[3,1]=x,y,z, result in matrix n°IndexPts1,2,3 bests // Mask is used to discard or considere the coordinates of an axes // for example: to get better approximation point only on x,y consideration in an object Matrix of x,y,z coordinates, // Use</pre>	

Mask=6 = %110 x is on, y is on and z is off

// Mask
= 5 =%101 will considere
x and z data to get a
better approximation
point index in the
objectM1

//modobjM
n°M1,n°PtLinkM,IndexP,V
ect,mod modify the
object M1, with its
PtLinkM according to mod
and Vect in point of
IndexP (in M1)

//
mod = 0
flattening on the plan
orthogonal to the vector
defined by mouse drag

//
1
exponential large by
mouse dragging

//
2
exponential narrow by
mouse dragging

//
3
sharp exponential by

mouse dragging	
<pre>//copyM n°M1,n°M2,mod copy M2 at the place defined by mod in M1, the resulting Matrix is M1 with a new size if necessary // The copy is made like: all the p values of M2 are copied to M1p; for all n M2(n,p) to M1(n,newposp) // mod<0 insert M2 in M1 at (-mod) position p // mod=0 : merge matrix M2 at the end of M1 resulting size is M1p+M2p // mod>0 : copy (replace data) at position p=mod</pre>	
<pre>//fftM0,1,2 Perform the fft of signal fct(t) in M0, the coeff ak and bk are output in M1 and M2. M0,M1 and</pre>	

M2 should have $M_n=1$. $N=M_0.p$ and $K_{max}=M_1.p=M_2.p$ //fftM0,1,2,a,b same as standard fft but taking window [a,b] // if $a=b=0$ same as fftM0,1,2	
---	--

divers

<i>action n</i>	Effectue différentes actions : <i>n=0 efface l'espace de tracé graphique</i> <i>n=1 désactive les boutons de contrôles</i> + - D * Z M correspondant à respectivement la dérivation (D) des fonctions tracés sur graphs, la récupérations x,y des points cliqués (*) sur l'espace graphique, le zoom (Z)(-) (+) et le déplacement (M) par la souris. <i>n=2 Actives les boutons de contrôles</i> évoqué ci-dessus. Par défaut les boutons de contrôle sont activés. L'usage de la souris (+-D*ZM) sur le graphe déplace le graphique, Le programme est relancé pour être recalculer et tracé dans les nouvelles limites du repère. <i>n=3 Change l'orientation de la lumière</i> <i>pour l'affichage 3D d'un objet</i> ou d'une fonction objet 3D (voir menu Math/3D function plot).
<i>clscmd</i>	<i>Efface la fenêtre de commande</i>

createbtn
btn,x,y,width,height,"BtnName"



key(0) renvoie la valeur du bouton appuyé.

Utiliser le test dans une boucle pour brancher vers un sous programme voir exemple:

```
createbtn1,360,50,40,20, "btn1"
```

```
createbtn2,360,50,40,20, "btn2"
```

```
...
```

```
createbtn9,360,50,40,20, "quit"
```

```
Loop :
```

```
a= key(0)
```

```
a== 1 => bsr program1
```

```
a== 2 => bsr program2
```

```
(...)
```

```
a==9 => goto quit
```

```
goto Loop
```

```
quit :
```

```
end
```

```
program1 :
```

```
...
```

```
rts
```

```
Program2 :
```

```
...
```

```
rts
```

Crée un bouton de numéro btn, à la position x,y sur la fenêtre CalcReg, de largeur width et de hauteur height. Le bouton sera créé avec le nom graphique bntname.

On peut créer 50 boutons maximum selon les presets de CalcReg.

Les boutons sont automatiquement effacés à la fin de l'exécution du programme.

On peut effacer un bouton en cours de programme par killbtn.

killbtn btn	<i>Effacer un bouton de numéro btn.</i>

```
//getserial Baudrate,SerFlag
```

```
//putserial Baudrate,SerFlag,bytedata
```

```
//key(0) or key(1) value from keyboard(0) or the mouse board  
rectangle(1)
```

```
// !! Use also key(0) to let the windows Update its MSG signals
```

```
// This is important to retrieve the Mouse up down moved, etc...
```

```
// put key(0) in a loop otherwise you won't get the proper reel  
time signals
```

```
//                                key(1) XpenDown,
```

```
//                                key(2)YpenDown,
```

```
//                                key(3)leftclickon=1, leftclickoff=0
```

```
//                                key(4)mousemoved=1,          not  
moved=0
```

```
//                                key(5)=XpenUp
```

```
//                                key(6)=YpenUp
```

```
//
```

V) Bugs connus

VI) Information et contact

